

Data Engineer Intern - Assignment

AMSTERDAM AIRBNB DATA ENGINEERING & ANALYTICS CHALLENGE

Professional Final Report

Author	PabodaFdo (Paboda Sathsarani Fernando)
Programme	BSc (Hons) Information Technology
Specialization	Data Science
Institution	Sri Lanka Institute of Information Technology (SLIIT)
Selected Market	Amsterdam, Netherlands
Submission Date	12 July 2026

Repository: <https://github.com/PabodaFdo/airbnb-data-engineering-challenge.git>

Table of Contents

Section	Page
1. Executive Summary	2
2. Objectives & Scope	3
3. Dataset Overview	4
4. Methodology	7
5. Engineering Approach	9
6. EDA Findings	13
7. Statistical Findings	21
8. Data Science Experiments	24
9. AI/ML Experiments	28
10. Visualizations	29
11. Business Recommendations	32
12. Cross-City Comparisons	34
13. Limitations & Caveats	35
14. Future Improvements	36
15. Reflection	37
Appendix A. AI Usage Disclosure	38
Appendix B. Reproducibility & Validation Evidence	39
References & Data Attribution	41

1. Executive Summary

This report presents a complete end-to-end data engineering and analytics workflow built using publicly available Inside Airbnb data for Amsterdam, Netherlands. The project follows a one-city, depth-first approach, with a strong focus on engineering quality, reproducibility, data-quality validation, meaningful analysis, and clear communication rather than attempting to cover too many optional areas superficially.

The workflow brings together data engineering, exploratory data analysis, statistical testing, a focused machine-learning experiment, continuous integration, and an interactive dashboard. Seven source datasets were processed through a structured pipeline that included raw-data verification, automated profiling, data-quality checks, a critical validation gate, cleaning and standardization, listing-level enrichment, Parquet-based processing, a DuckDB analytical warehouse, SQL analysis, exploratory analysis, statistical hypothesis testing, a focused price-prediction experiment, automated testing, GitHub Actions continuous integration, and a deployed Streamlit dashboard.

The final pipeline preserved all 10,465 canonical listings, matched 10,369 detailed listing records, and retained 96 summary-only listings that would otherwise have been lost if an inner join had been used. It also processed 545,162 review records and 3,819,725 calendar rows.

The source-level validation stage produced 83 PASS results, 7 WARNING results, and no failures. Warehouse validation completed with 17 PASS results and 0 failures, while all 13 focused automated tests passed successfully. Depending on system conditions and disk caching, the full pipeline completed in approximately one to two minutes on a local Windows machine with 8 GB of RAM.

Metric	Result
Canonical listings preserved	10,465
Detailed listings matched	10,369
Summary-only listings retained	96
Review events reconciled	545,162
Calendar rows reconciled	3,819,725
Source validation	83 PASS / 7 WARNING / 0 FAIL
Warehouse validation	17 PASS / 0 FAIL
Automated tests	13 passed / 0 failed
GitHub Actions CI	Passing
Best predictive model	Random Forest Regressor
Best model performance	MAE €78.61 RMSE €133.76 R ² 0.5884
Live dashboard	amsterdam-airbnb-market-explorer.streamlit.app
Full pipeline runtime	Approximately 1-2 minutes on the local 8 GB RAM Windows environment

The analysis revealed several meaningful business patterns. Entire homes and apartments are priced considerably higher than private rooms, with a large rank-biserial effect size of 0.697, showing that room type is a major factor in pricing differences. Neighbourhood and accommodation capacity also play an important role in segmenting the market and influencing listing prices.

In contrast, although superhost listings achieved statistically different review scores compared with non-superhost listings, the actual practical difference was small. This highlights the importance of looking beyond statistical significance and considering whether a result has meaningful real-world impact.

The focused price-prediction experiment compared three models: a Dummy Regressor, Ridge Regression, and Random Forest Regressor. Among them, the Random Forest model produced the best performance on the held-out test set, achieving an MAE of €78.61, an RMSE of €133.76, and an R^2 score of 0.5884.

These validated analytical and machine-learning outputs were then made available through a seven-tab interactive Streamlit dashboard, deployed on Streamlit Community Cloud for easier exploration and presentation of the project findings.

The key takeaway is that reliable analytics and machine-learning experiments depend on getting the fundamentals right. This includes working at the correct data grain, using safe join strategies, handling missing values honestly, defining clear validation severity levels, preventing data leakage, and using proxy measures carefully without overstating what they represent.

This project applies these principles throughout the full workflow, from raw data processing and validation to analysis, predictive modelling, and the delivery of results through a live interactive dashboard.

2. Objectives & Scope

The main objective of this project was to build a reliable and reproducible data engineering workflow that transforms raw Amsterdam Airbnb data into validated, analysis-ready datasets and a structured analytical warehouse. These outputs were then used for focused exploratory data analysis, statistical testing, and evidence-based business recommendations.

2.1 Core Objectives

The project was designed to achieve the following objectives:

- Understand and document the structure, grain, keys, relationships, and limitations of all seven source datasets.
- Automate dataset profiling and source-level data-quality checks.
- Apply context-aware cleaning methods without inventing values for missing data or modifying the original raw files.
- Preserve the complete canonical listing population while enriching it with detailed listing information, review summaries, and calendar-based aggregates.
- Use memory-efficient processing methods that can run reliably on a Windows laptop with 8 GB of RAM.
- Build a lightweight analytical model using DuckDB and create reusable SQL queries for further analysis.
- Conduct focused exploratory data analysis and two statistical hypothesis tests, including interpretation of effect sizes.
- Translate technical and analytical findings into clear business recommendations that can be understood by both technical and non-technical stakeholders.

2.2 Selected Market and Prioritization Rationale

Amsterdam, Netherlands, was selected as the sole market for this project. This was a deliberate decision based on the limited execution time of three days and the need to maintain a high standard of engineering quality.

Expanding the project to multiple cities would have introduced additional complexity, including more data downloads, schema harmonization, currency differences, cross-city validation rules, and a larger reporting scope. Under the limited execution window, this would have reduced the time available for important areas such as data validation, testing, documentation, statistical analysis, and overall report quality.

For this reason, the project followed a **depth-over-breadth** strategy. The focus was on fully profiling one market, building a repeatable pipeline, applying explicit validation rules, cleaning data carefully, performing safe listing-level enrichment, creating and validating a DuckDB analytical warehouse, answering eight analytical questions through SQL, producing ten EDA visualizations, conducting two statistical hypothesis tests, adding automated tests, and completing the required engineering documentation.

2.3 Scope Included

- Dataset familiarization for all seven source files.

- Automated profiling and data-quality reporting.
- PASS / WARNING / FAIL validation framework with a critical validation gate.
- Price, date, Boolean, key, and missing-value handling.
- Listing-level enrichment with review and calendar aggregates.
- Parquet processed layer and DuckDB analytical warehouse.
- Eight business-oriented analytical SQL queries.
- Ten EDA visualizations.
- Two statistical hypotheses with effect sizes.
- Focused price-prediction experiment comparing three regression models.
- Random Forest feature-importance and actual-vs-predicted diagnostic analysis.
- Thirteen automated tests and GitHub Actions continuous integration.
- Interactive seven-tab Streamlit dashboard with live cloud deployment.
- Assumptions, decision log, completed work, incomplete work, AI disclosure, and submission-details documentation.

2.4 Scope Intentionally Deferred

The focused machine-learning experiment, GitHub Actions continuous integration, interactive Streamlit dashboard, and live dashboard deployment were completed and are therefore no longer deferred. The work intentionally left for future extensions includes multi-city execution and cross-city comparison, full cloud-native pipeline deployment, workflow orchestration, incremental ingestion, Dockerization, advanced geospatial analysis, extensive cross-validation and hyperparameter optimization, production model serving, and full operational observability.

3. Dataset Overview

The project uses seven Amsterdam files from Inside Airbnb. The files represent listing summaries, detailed listing attributes, calendar availability, review events, neighbourhood metadata, and geographic boundaries. Raw source files are treated as immutable inputs.

File	Rows / Features	Columns / Properties	Grain
neighbourhoods.csv	22	2	One row per neighbourhood
listings.csv	10,465	19	One row per canonical listing
reviews.csv	545,162	2	Review event by listing and date
listings.csv.gz	10,369	90	One row per detailed listing
neighbourhoods.geojson	22 features	Geometry + properties	One feature per neighbourhood
calendar.csv.gz	3,819,725	5	One row per listing per calendar date
reviews.csv.gz	545,162	6	One row per individual review

Table 1. Amsterdam source datasets and their analytical grain

3.1 Canonical Listing Population

The summary listings file, `listings.csv`, is used as the canonical source for the Amsterdam listing population because it contains 10,465 listings, compared with 10,369 records in the detailed listings dataset. To avoid unnecessary data loss, the 96 listings that appear only in the summary file are preserved rather than being removed through an inner join. Where detailed information is unavailable for these listings, the corresponding attributes are intentionally left as null values instead of being artificially filled or inferred.

3.2 Candidate Keys and Relationships

Entity / Relationship	Key	Role
Listing	<code>listings.csv.id</code>	Canonical identifier
Detailed listing	<code>listings.csv.gz.id</code> → <code>listings.csv.id</code>	Detailed attribute enrichment
Detailed review	<code>reviews.csv.gz.id</code>	Individual review identifier
Review-to-listing	<code>reviews.csv.gz.listing_id</code> → <code>listings.csv.id</code>	Many-to-one relationship
Calendar-to-listing	<code>calendar.csv.gz.listing_id</code> → <code>listings.csv.id</code>	Many-to-one relationship
Neighbourhood	neighbourhood name	Reference and geographic enrichment

Table 2. Key entities, candidate keys, and cross-dataset relationships

3.3 Important Dataset Limitations

- 3,994 of 10,465 summary listings and 3,992 of 10,369 detailed listings have missing price values.
- A missing price is preserved as null and is never interpreted as zero.
- The detailed listing source does not contain all canonical summary listings.
- The summary review file does not have a unique review identifier.
- Repeated (`listing_id`, `date`) combinations in the summary review data are not automatically duplicate events.
- Calendar unavailability does not reveal whether a date was booked, host-blocked, under maintenance, or unavailable for another reason.
- Review counts are not complete booking counts because not every guest leaves a review.
- The data is observational, so statistical findings should not be presented as causal evidence.

3.4 Data Familiarization Findings

The familiarization stage identified source differences before any transformation was attempted. This prevented several common errors: treating every repeated review date as a duplicate, converting missing prices to zero, assuming the detailed listing source was complete, and describing unavailable calendar dates as verified occupancy.

Metric	Result
Canonical summary listings	10,465
Detailed listing records	10,369
Summary-only listings	96
Detailed review events	545,162
Calendar rows	3,819,725
Amsterdam neighbourhoods	22
Repeated summary review (listing_id, date) extra rows	22,541
Missing detailed reviewer names	1

These findings directly influenced the engineering design. The canonical listing grain was preserved through left-based enrichment; review and calendar sources were aggregated before joining; missing values were handled contextually; and proxy terminology was used when the source data could not support stronger claims.

4. Methodology

The overall methodology follows a staged data engineering architecture in which each layer has a distinct responsibility. Validation occurs before cleaning and enrichment, and critical failures block downstream processing. This design improves traceability and prevents analytical results from being built on structurally invalid inputs.



Figure 1. Amsterdam Airbnb Data Engineering Architecture

The methodology begins with raw-input verification to confirm the presence of all seven required source files. Automated profiling then generates compact dataset- and column-level summaries. Validation applies explicit source-specific rules and records PASS, WARNING, or FAIL outcomes. A critical gate stops the pipeline if a FAIL is present.

Cleaning standardizes prices, dates, Boolean values, and key integrity while preserving source limitations. Enrichment starts from the 10,465-listing canonical population and joins only compact listing-level aggregates from review and calendar sources. Processed outputs are stored in Parquet and loaded into a DuckDB analytical warehouse. SQL, EDA, statistical tests, the price-prediction experiment, and the Streamlit dashboard consume validated analytical outputs rather than raw multi-million-row sources.

4.1 Analytical Method Selection

EDA was intentionally focused on interpretable business questions rather than a large number of unconnected plots. The selected analyses examine supply concentration, neighbourhood pricing, room-type segmentation, review activity, calendar unavailability patterns, accommodation capacity, price skew, and feature correlations.

For statistical testing, the Mann-Whitney U test was used for two independent-group comparisons. This non-parametric approach is suitable when distributions are skewed, bounded, or affected by outliers and when normality should not be assumed. Statistical significance was interpreted together with rank-biserial effect size, group medians, sample size, practical importance, and business context.

For predictive experimentation, the project used an 80/20 held-out train-test split with fixed `random_state=42`, explicit target-leakage prevention, numerical and categorical preprocessing, a `log1p` target transformation, and three models: Dummy Regressor, Ridge Regression, and Random Forest Regressor. Performance was evaluated in euros using MAE and RMSE, together with R^2 for explained variation.

4.2 Business Interpretation Framework

Finding → Business Meaning → Recommended Action

Every major analytical figure is interpreted using this structure. The goal is to avoid presenting charts as isolated technical artifacts and instead explain what changed, why it matters, what decision could follow, and what caveat limits the strength of the conclusion.

5. Engineering Approach

The engineering approach is intentionally simple enough to run locally while still demonstrating production-minded principles: modular stages, deterministic paths, immutable raw inputs, automated profiling, explicit validation severity, a fail-fast gate, one-row-per-listing grain protection, columnar processed storage, analytical modeling, logging, error handling, and automated integrity tests.

5.1 8 GB RAM-Aware Processing Strategy

The target development machine has 8 GB RAM. Instead of downsampling core datasets or moving immediately to distributed infrastructure, the pipeline uses a hybrid Pandas and DuckDB design. Pandas handles small and medium tables; DuckDB handles the large calendar and detailed review files and performs aggregation without materializing unnecessary full DataFrames.

Area	Choice	Reason
Small / medium files	Pandas	Simple, readable in-memory transformations
calendar.csv.gz	DuckDB	3.82M rows; aggregate before downstream joins
reviews.csv.gz	DuckDB	545k review events; avoid carrying free text into listing-level analytics
Processed layer	Parquet	Compact columnar storage with type preservation
Join strategy	Aggregate first	Protect grain and reduce memory pressure
Execution pattern	Sequential stages	Release large temporary objects between stages

Table 3. Memory-aware processing decisions

5.2 Automated Profiling

Reusable profiling logic is implemented in `src/profile.py`. The profiler processes all seven datasets and writes compact reports rather than returning complete raw tables. Metrics include row count, column count, duplicate count, data types, missing values, missing percentages, cardinality, ranges where meaningful, and representative samples.

Metric	Result
Datasets profiled successfully	7 / 7
Dataset-level output	outputs/data_quality/dataset_summary.csv
Column-level output	outputs/data_quality/column_profile.csv

This stage improves reproducibility because profiling is no longer dependent on manual notebook inspection alone. The same profiler can be executed again after source updates, and the outputs can be compared over time.

5.3 Data-Quality Validation and Critical Gate

Metric	Result
PASS	83
WARNING	7
FAIL	0

Validation rules cover required columns, missing identifiers, unique-key expectations, duplicate IDs, missing and negative prices, geographic ranges, availability ranges, room-type categories, date parsing, foreign-key coverage, source-coverage differences, missing metadata, and repeated review rows.

FAIL exists → STOP | No FAIL → CONTINUE

The validation gate allows known and documented warnings to continue, but it prevents downstream processing when a critical FAIL result exists. This design separates source imperfection from structural invalidity instead of pretending that all data-quality issues are equally severe.

5.4 Cleaning and Standardization

Cleaning logic is implemented in `src/clean.py`. The stage parses currency-formatted prices, preserves raw price values, parses relevant dates, standardizes Boolean fields, and validates listing-key integrity. The raw layer is never overwritten.

- Missing price is preserved as null, not imputed as zero.
- Invalid dates are parsed safely and invalid values become missing timestamps rather than fabricated dates.
- Boolean values are normalized from common Airbnb representations.
- Unique and non-null listing identifiers are verified after cleaning.
- Cleaned outputs are written to Parquet for efficient downstream access.

Metric	Result
cleaned_listings.parquet	10,465 rows; unique non-null listing IDs
cleaned_detailed_listings.parquet	10,369 rows; unique non-null listing IDs

5.5 Listing-Level Enrichment

The enrichment stage starts from `listings.csv` as the canonical population. Detailed listing attributes are joined using a left-preserving strategy. Reviews and calendar data are aggregated to one row per listing before they are joined. This prevents row multiplication and avoids creating enormous many-to-many intermediate tables.

Metric	Result
Canonical listings preserved	10,465
Detailed-source matches	10,369
Summary-only listings retained	96
Final grain	One row per canonical listing
Listing ID integrity	Unique and non-null

5.6 Processed Data Layer

The processed layer contains cleaned listing tables, listing-level review aggregates, listing-level calendar aggregates, and the final enriched listing master. Parquet was selected because it preserves data types, supports column pruning, reduces storage overhead, and integrates well with both Pandas and DuckDB.

- data/processed/cleaned_listings.parquet
- data/processed/cleaned_detailed_listings.parquet
- data/processed/review_listing_aggregates.parquet
- data/processed/calendar_listing_aggregates.parquet
- data/processed/enriched_listing_master.parquet

5.7 DuckDB Analytical Warehouse

The analytical warehouse is built by src/build_warehouse.py and stored at data/warehouse/airbnb_analytics.duckdb. The model is intentionally lightweight: enough structure for reusable analytical SQL without overengineering a take-home assignment into a complex enterprise warehouse.

- enriched_listing_master
- dim_listings
- dim_neighbourhoods
- fact_review_activity
- fact_calendar_activity

Metric	Result
Canonical listings reconciled	10,465
Review events reconciled	545,162
Calendar rows reconciled	3,819,725
Warehouse validation	17 PASS / 0 FAIL

5.8 Analytical SQL

Eight business-oriented SQL queries were written against the DuckDB analytical model. The queries examine pricing and performance by room type, neighbourhood pricing and activity, host portfolio concentration, superhost versus non-superhost performance, neighbourhood review activity, highly reviewed but lower-rated listings, availability-based proxy patterns, and pricing by accommodation capacity.

1. Pricing and performance by room type
2. Neighbourhood pricing and market activity
3. Host portfolio concentration and supply share
4. Superhost versus non-superhost performance
5. Neighbourhood review activity
6. Highly reviewed but relatively low-rated listings
7. Availability-based proxy patterns by neighbourhood
8. Pricing by accommodation capacity

5.9 Automated Testing and Reproducibility

The focused automated test suite covers price parsing, raw-price preservation, null preservation, valid and invalid date parsing, Boolean normalization, key integrity, duplicate rejection, derived features, and final canonical-grain preservation. Python compilation and all 13 tests completed successfully.

Metric	Result
Tests passed	13

Tests failed	0
Test command	<code>python -m pytest tests/ -v</code>
Compilation command	<code>python -m compileall -q run_pipeline.py src tests experiments dashboard</code>
Pipeline command	<code>python run_pipeline.py --city amsterdam</code>
Full pipeline runtime	Approximately 1-2 minutes depending on system conditions and disk caching

5.10 GitHub Actions Continuous Integration

A GitHub Actions workflow automatically verifies the project on relevant pushes and pull requests. The workflow checks out the repository, sets up Python 3.13, installs dependencies, compiles project modules, and runs the automated tests. The workflow has completed successfully on the dev branch, providing repeatable verification beyond local execution.

CI Stage	Status
Checkout repository	Completed
Set up Python 3.13	Completed
Install dependencies	Completed
Compile Python modules	Completed
Run automated tests	Passing

5.11 Interactive Dashboard and Live Deployment

The final project includes a Streamlit and Plotly dashboard that reads validated processed outputs rather than the large raw source files. It contains seven analytical tabs: Overview, Market Explorer, Pricing, Reviews & Availability, Statistics, Machine Learning, and Data Engineering. Sidebar filters cover neighbourhood, room type, host portfolio segment, superhost status, available price range, and guest capacity.

The default dashboard preserves the full canonical listing population. Missing price and capacity values remain preserved by default; they are excluded from filtered results only when the corresponding numeric range is actively narrowed. The app is deployed on Streamlit Community Cloud at <https://amsterdam-airbnb-market-explorer.streamlit.app/>.

Dashboard Metric	Result
Tabs	7
Primary dataset	<code>data/processed/enriched_listing_master.parquet</code>
Core interaction	Six sidebar filter dimensions
Deployment	Streamlit Community Cloud
Live URL	https://amsterdam-airbnb-market-explorer.streamlit.app/

6. EDA Findings

Ten EDA visualizations were produced. This section presents eight primary figures selected for analytical strength, business relevance, and interpretability. Two additional supporting figures are included later in the visualization section.

6.1 Listings by Neighbourhood

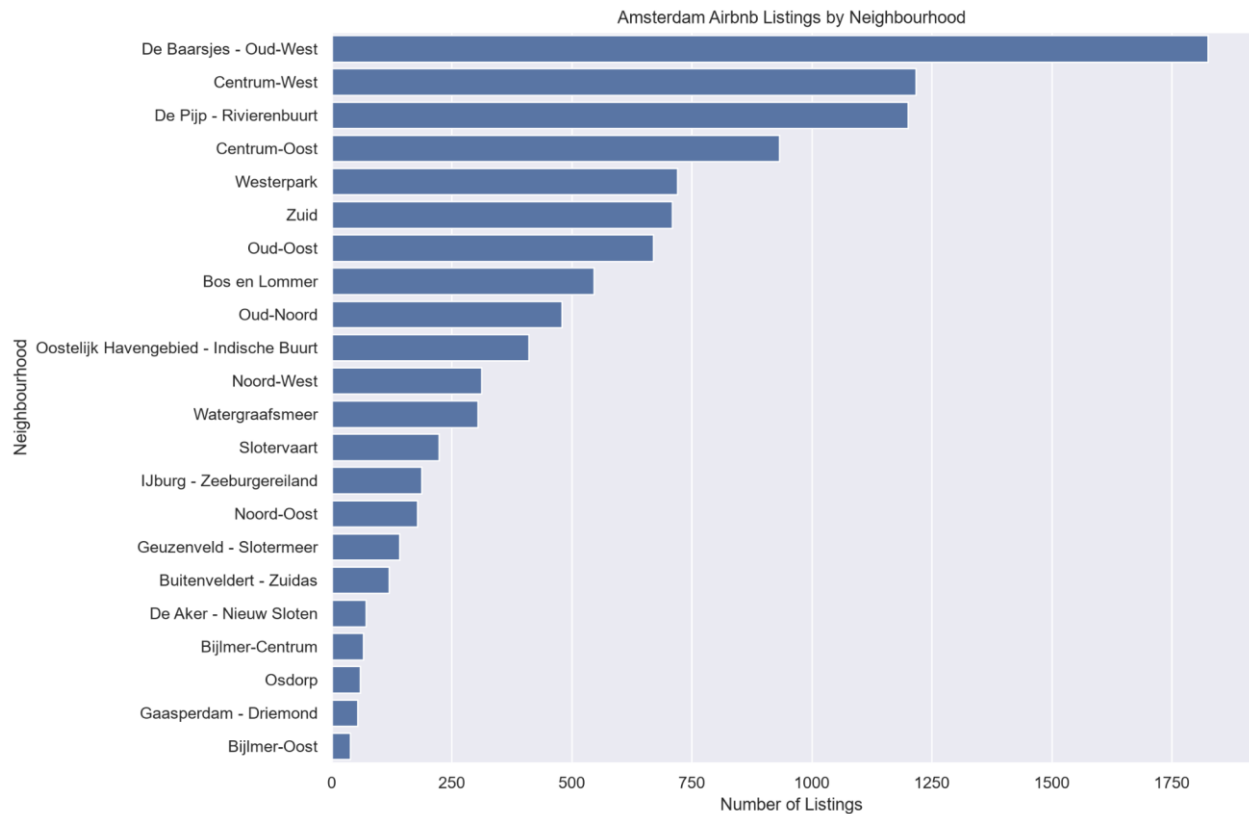


Figure 2. Amsterdam Airbnb Listings by Neighbourhood

Finding	Airbnb supply is highly concentrated in a relatively small number of neighbourhoods. De Baarsjes - Oud-West has the largest listing count, followed by Centrum-West and De Pijp - Rivierenbuurt.
Business meaning	Competitive intensity and available supply differ materially by neighbourhood. City-wide benchmarks can therefore hide local market structure.
Recommended action	Benchmark listings within the same neighbourhood and room-type segment before making pricing or market-entry decisions.
Caveat	Listing count measures supply, not demand or profitability.

6.2 Median Price by Neighbourhood

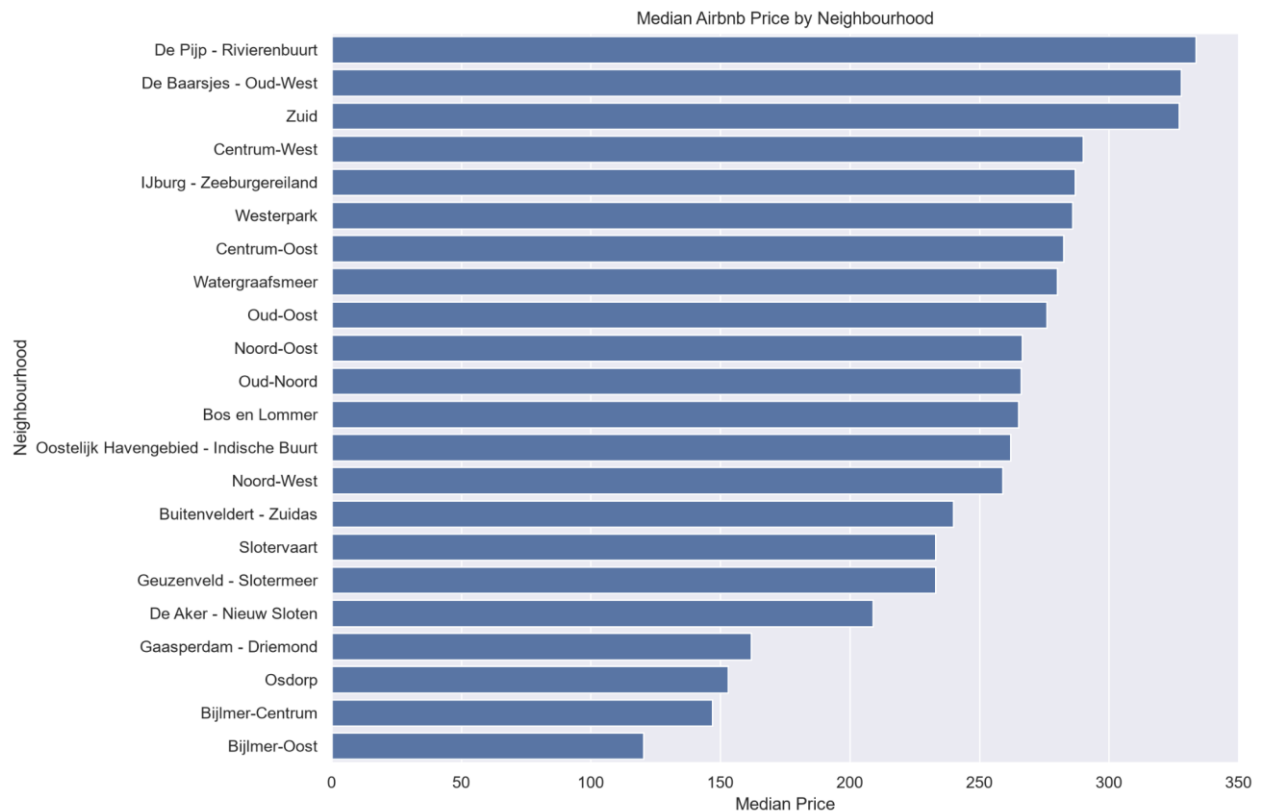


Figure 3. Median Airbnb Price by Neighbourhood

Finding	Median prices vary substantially across Amsterdam. De Pijp - Rivierenbuurt, De Baarsjes - Oud-West, and Zuid appear among the highest-priced neighbourhoods, while several outer areas have much lower medians.
Business meaning	Location is strongly associated with pricing power, so one city-wide benchmark is too coarse for fair comparison.
Recommended action	Use neighbourhood-specific pricing benchmarks and further segment by room type and capacity.
Caveat	Neighbourhood alone does not cause the observed differences; property mix and other listing characteristics also matter.

6.3 Median Price by Room Type



Figure 4. Median Price by Room Type

Finding	Entire homes/apartments have the highest median price, at roughly €330, followed by hotel rooms, private rooms, and shared rooms.
Business meaning	Room type is one of the clearest pricing segmentation variables in the Amsterdam market.
Recommended action	Do not compare all listings together. Price benchmarks should be separated by room type and ideally neighbourhood.

6.4 Review Activity by Neighbourhood

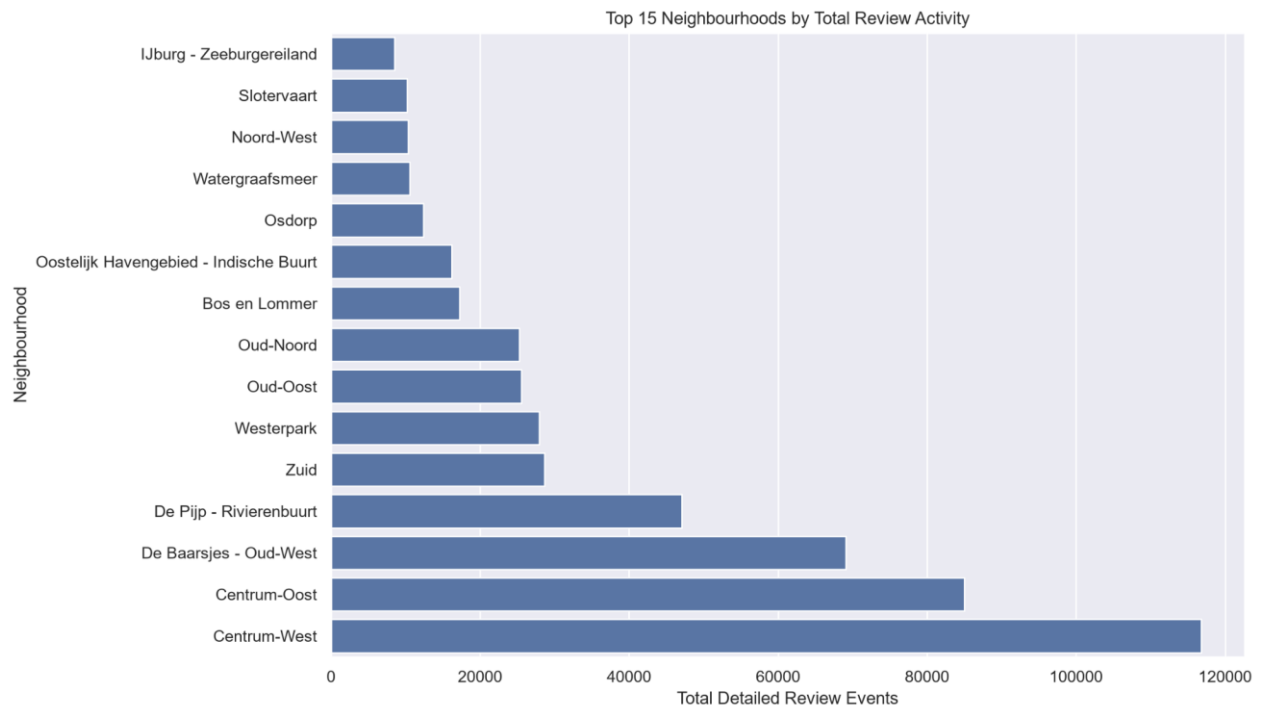


Figure 5. Top Neighbourhoods by Total Detailed Review Activity

Finding	Historical detailed review activity is concentrated in central and high-supply neighbourhoods, especially Centrum-West, Centrum-Oost, De Baarsjes - Oud-West, and De Pijp - Rivierenbuurt.
Business meaning	Some neighbourhoods show much stronger historical guest-review activity than others, but total review volume is influenced by supply, listing age, and guest behaviour.
Recommended action	Use review activity as a contextual engagement indicator and compare it alongside listing counts or per-listing rates.
Caveat	Review count is not booking count.

6.5 Unavailability Proxy by Neighbourhood

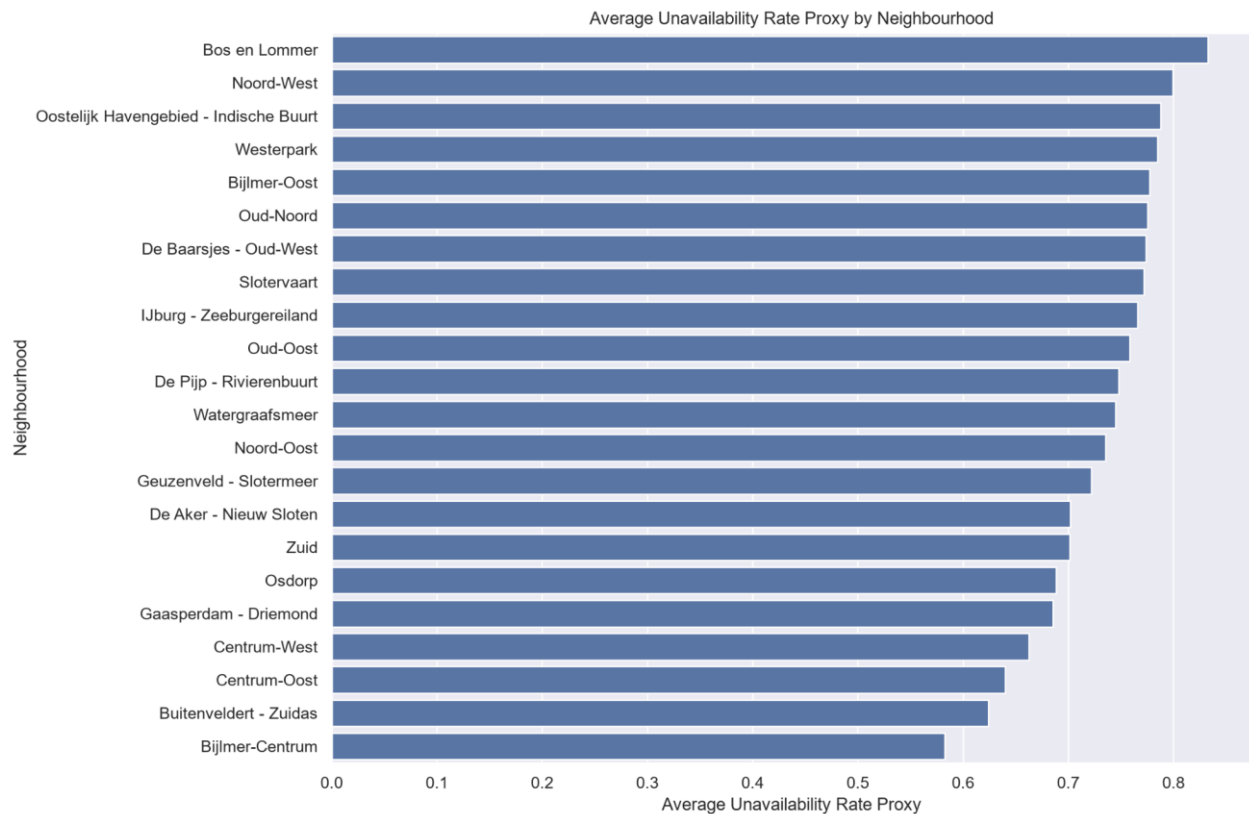


Figure 6. Average Unavailability Rate Proxy by Neighbourhood

Finding	Average calendar unavailability differs across neighbourhoods, with Bos en Lommer among the highest proxy values and several neighbourhoods showing materially lower values.
Business meaning	Calendar availability patterns vary geographically, but the source does not reveal why dates are unavailable.
Recommended action	Use the metric only as an availability-based proxy and never report it as verified occupancy.
Caveat	An unavailable date may represent a booking, host block, maintenance, personal use, regulation, or another reason.

6.6 Median Price by Accommodation Capacity

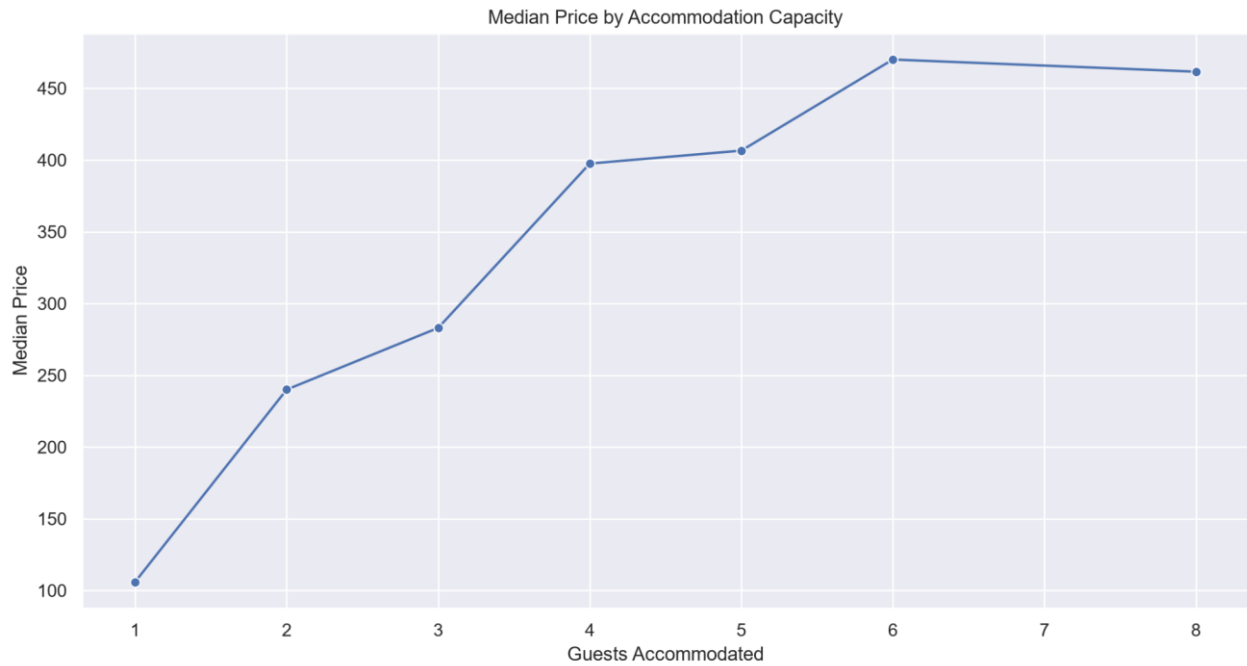


Figure 7. Median Price by Accommodation Capacity

Finding	Median price generally rises with accommodation capacity, increasing from roughly €105 for one guest to around €470 for six guests before flattening slightly at higher capacities.
Business meaning	Guest capacity is an important pricing dimension and helps explain part of the observed market segmentation.
Recommended action	Compare listings against properties with similar capacity rather than relying on city-wide averages.
Caveat	Interpret the small decline at eight guests cautiously without first checking group sample sizes.

6.7 Price Distribution Excluding IQR Upper Outliers

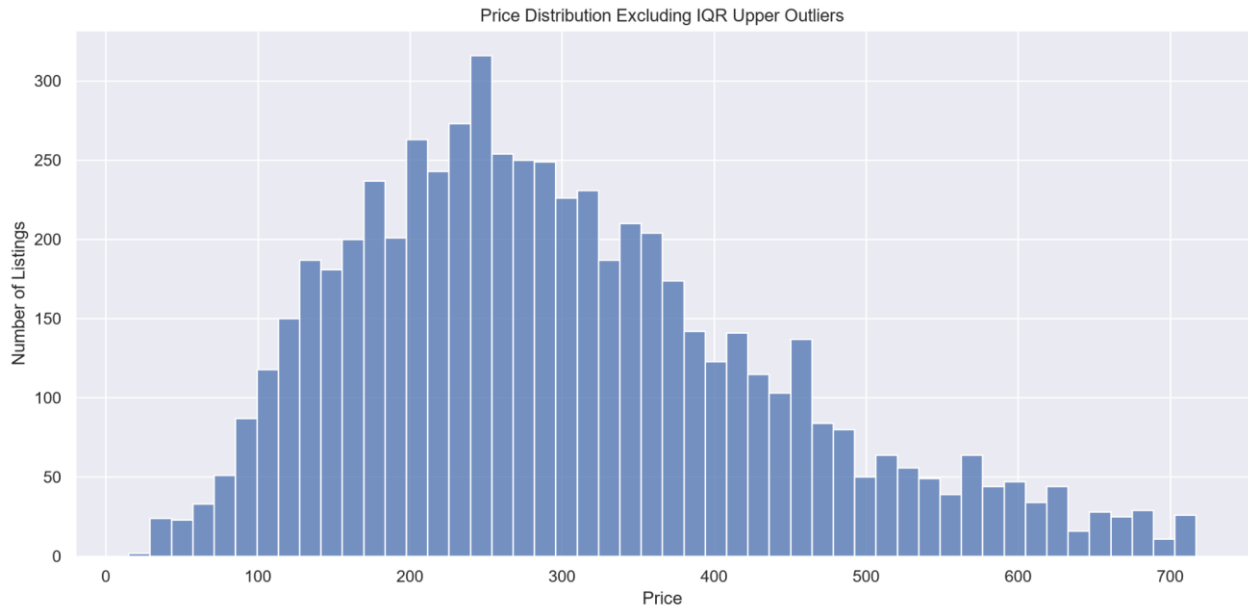


Figure 8. Price Distribution Excluding IQR Upper Outliers

Finding	After excluding upper IQR outliers for visualization, the price distribution remains right-skewed, with most valid prices concentrated in the lower-to-middle hundreds and a long premium tail.
Business meaning	Means can be heavily influenced by premium listings, so medians are often more representative for broad market benchmarking.
Recommended action	Use median-based summaries and segment comparisons by room type, neighbourhood, and capacity.
Caveat	Outliers were excluded only from this visualization; the underlying source and processed datasets were not modified or stripped of those records.

6.8 Correlation Matrix

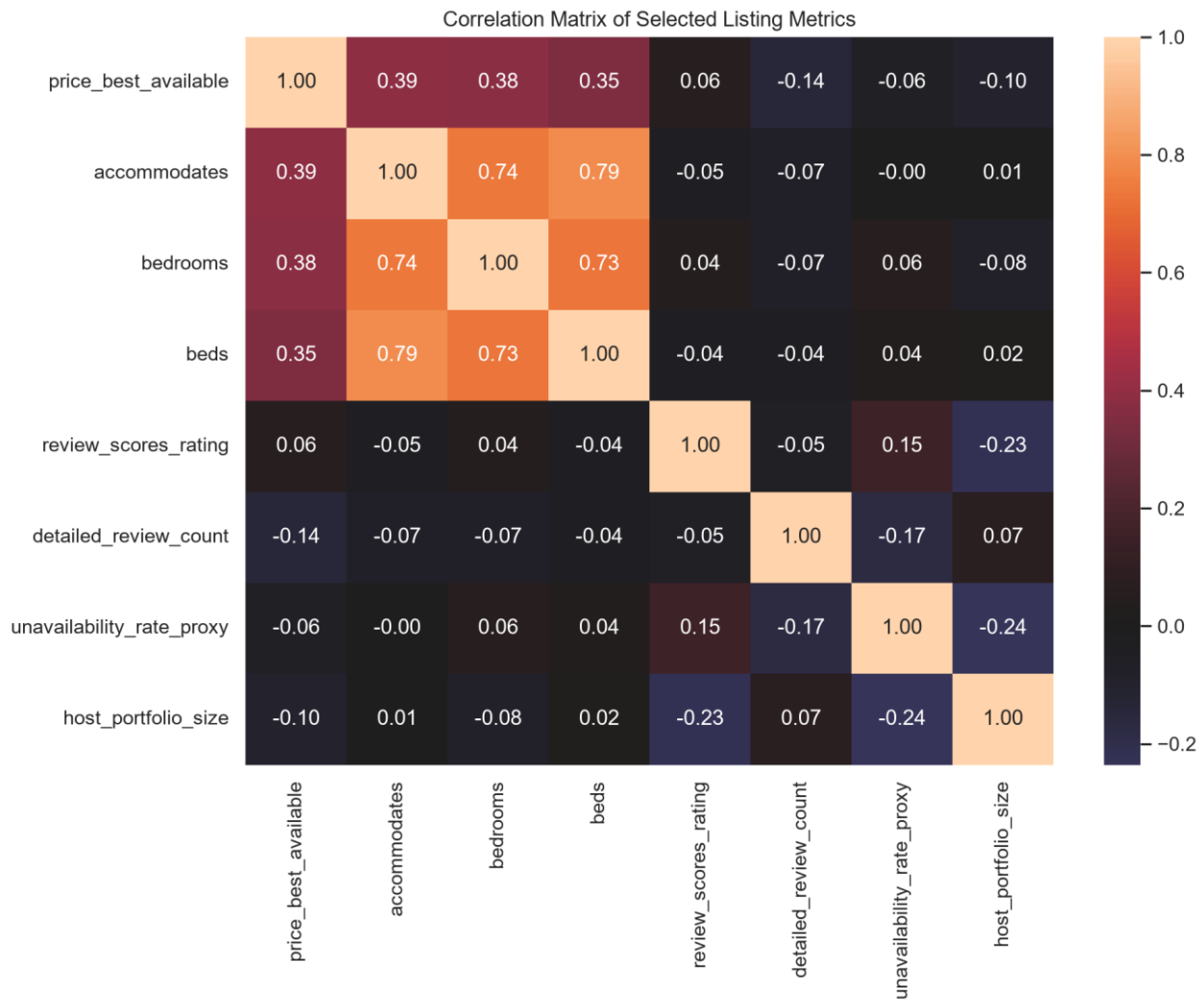


Figure 9. Correlation Matrix of Selected Listing Metrics

Finding	The strongest correlations are among accommodates, bedrooms, and beds. Price has moderate positive correlations with these size-related variables, while review score, detailed review count, and unavailability proxy show much weaker linear relationships with price.
Business meaning	Accommodation size is meaningfully associated with price, but several popular marketplace metrics have limited linear explanatory value on their own.
Recommended action	Include size-related features in future pricing analysis, but handle highly correlated variables carefully to avoid redundant predictors.
Caveat	Correlation does not imply causation, and weak linear correlation does not rule out nonlinear relationships.

7. Statistical Findings

Two hypotheses were selected to complement the exploratory analysis. The focus was on completing a small number of tests rigorously rather than producing many shallow p-values. Both tests use the Mann-Whitney U method and report group sizes, medians, p-values, rank-biserial effect sizes, business interpretation, and limitations.

7.1 Hypothesis 1 - Entire Homes/Apartments vs Private Rooms

Research question: Do prices differ between entire homes/apartments and private rooms?

H0: The price distributions of entire homes/apartments and private rooms do not differ meaningfully.

H1: The price distributions of entire homes/apartments and private rooms differ.

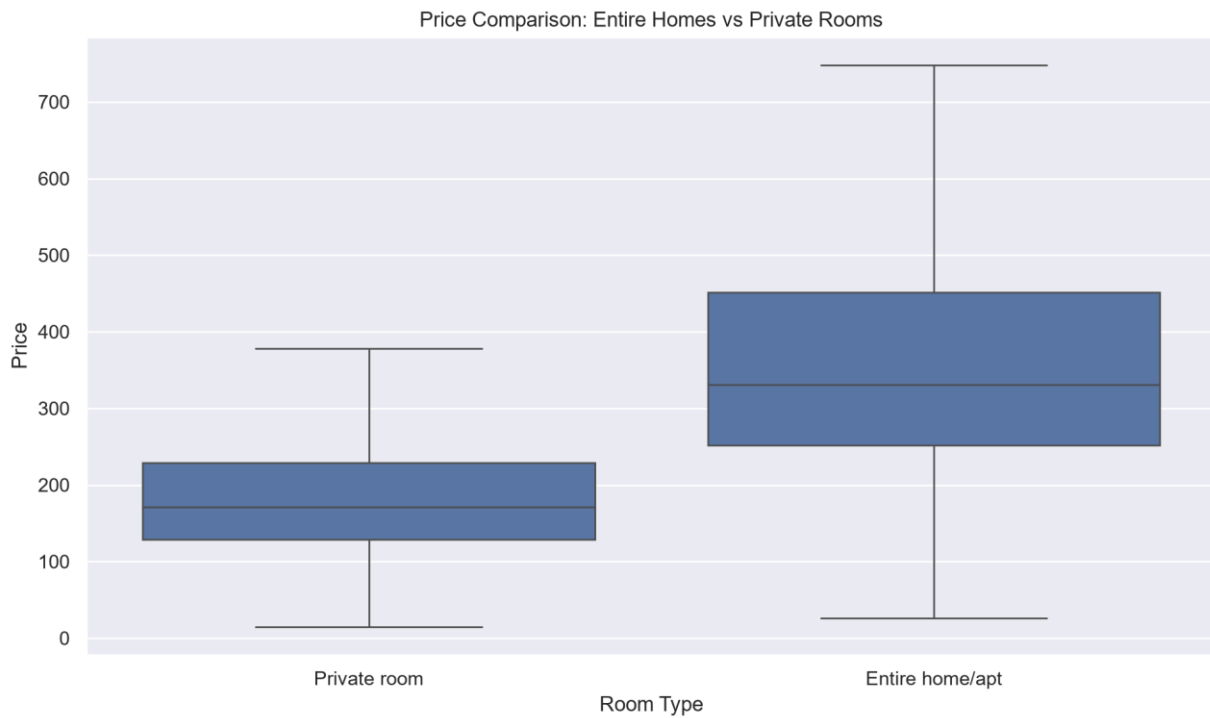


Figure 10. Price Distribution Comparison: Entire Homes/Apartments vs Private Rooms

Metric	Result
Test	Mann-Whitney U
Entire home/apt sample size	4,771
Entire home/apt median	€331
Private room sample size	1,653
Private room median	€171
Mann-Whitney U statistic	6,692,891.0
P-value	< 0.001
Rank-biserial effect size	0.697
Decision	Reject H0

The median price difference is €160: €331 for entire homes/apartments versus €171 for private rooms. The test rejects H0 and the rank-biserial effect size of 0.697 is large, so the difference is not only statistically detectable but also practically meaningful.

Finding	Entire homes/apartments are substantially more expensive than private rooms.
Business meaning	Room type is a major pricing segmentation factor and mixing room types can make city-wide benchmarks misleading.
Recommended action	Build pricing benchmarks separately by room type and then refine them by neighbourhood and accommodation capacity.
Caveat	The test identifies a distributional difference but does not prove that room type alone causes the price gap.

7.2 Hypothesis 2 - Superhosts vs Non-Superhosts

Research question: Do review scores differ between superhost and non-superhost listings?

H0: The review-score distributions of superhost and non-superhost listings do not differ meaningfully.

H1: The review-score distributions of superhost and non-superhost listings differ.

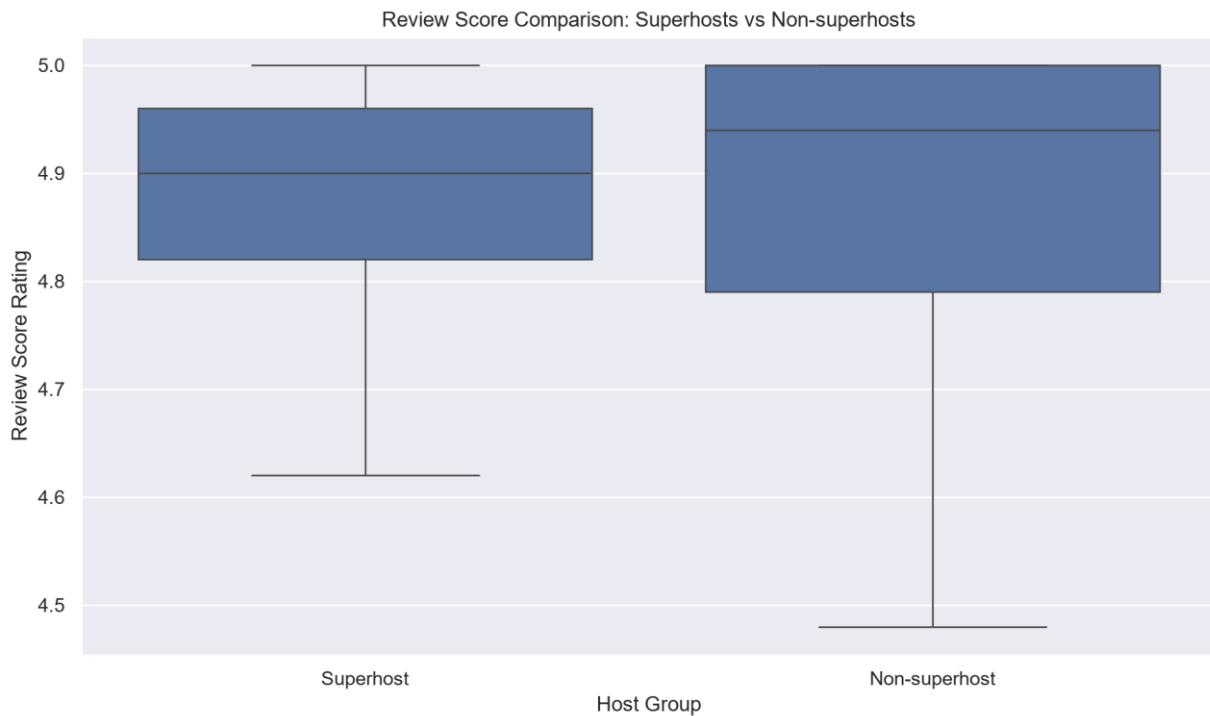


Figure 11. Review Score Distribution Comparison: Superhosts vs Non-Superhosts

Metric	Result
Test	Mann-Whitney U
Superhost sample size	1,661
Superhost median review score	4.90
Non-superhost sample size	7,566
Non-superhost median review score	4.94

Metric	Result
Mann-Whitney U statistic	5,293,896.5
P-value	3.19e-25
Rank-biserial effect size	-0.158
Decision	Reject H0

The test rejects H0, but the practical difference is small. Superhosts have a median review score of 4.90, compared with 4.94 for non-superhosts, a difference of only 0.04 points. The effect size of -0.158 is small. This is a strong example of why statistical significance should not be interpreted as automatic business importance.

Finding	The two review-score distributions differ statistically, but the practical effect is small and the groups overlap heavily.
Business meaning	Large sample sizes can make very small differences statistically detectable even when the business effect is limited.
Recommended action	Do not use superhost status as the sole indicator of listing quality; combine it with review volume, response behaviour, consistency, and listing characteristics.
Caveat	The analysis is observational and superhost status depends on platform criteria beyond review score alone.

7.3 Statistical Conclusions in Business Terms

Hypothesis	Decision	Effect Size	Magnitude	Business Interpretation
H1: Entire homes vs private rooms	Reject H0	0.697	Large	Strong practical pricing difference
H2: Superhosts vs non-superhosts	Reject H0	-0.158	Small	Statistically significant but limited practical difference

Table 4. Summary of statistical findings and practical significance

The two hypotheses together provide a balanced statistical story. Room type produces a large and practically meaningful pricing difference, whereas superhost status shows that a very small effect can still be statistically significant in a large sample. This distinction is important for decision-makers who might otherwise overreact to p-values without considering effect size and actual group differences.

8. Data Science Experiments

A focused price-prediction experiment was completed after the core engineering pipeline, warehouse, EDA, statistical analysis, automated testing, continuous integration, and documentation were stabilized. The goal was not to build a production pricing engine, but to test whether the enriched listing-level dataset contains useful predictive signal under a reproducible and leakage-aware design.

8.1 Research Question and Modeling Dataset

Research question: Can listing characteristics predict the available Airbnb listing price for Amsterdam listings?

The experiment uses data/processed/enriched_listing_master.parquet, which contains one row per canonical listing. Only listings with a valid positive target price were included. Missing target values were excluded rather than imputed or fabricated.

Metric	Count
Total canonical listings	10,465
Listings with valid positive target price	6,471
Listings excluded because target price was missing or invalid	3,994
Training observations	5,176
Test observations	1,295

8.2 Target Variable and Leakage Prevention

The target variable is price_best_available. To prevent target leakage, all fields derived directly from price were excluded from the feature set: price_raw, price, detailed_price, detailed_price_raw, price_best_available, price_source, price_per_bedroom, and price_per_guest.

Leakage principle: the model must not receive any field that directly encodes or derives from the target price.

8.3 Predictors and Preprocessing

The feature set combines location, property size, room and property type, host attributes, review activity, review scores, availability, and portfolio characteristics. Categorical features include neighbourhood, room type, detailed property type, host superhost status, host identity verification, and host portfolio segment. Numerical features include latitude, longitude, accommodates, bathrooms, bedrooms, beds, minimum and maximum nights, availability_365, review counts, reviews per month, review scores, host listing counts, review-events-per-active-year, and unavailability_rate_proxy.

- Median imputation for missing numerical features.
- Explicit missing-category handling for categorical features.
- One-hot encoding with unknown-category protection and minimum frequency handling.
- Numerical feature standardization.
- log1p target transformation with inverse transformation back to euros for evaluation.
- Fixed random_state=42 and an 80/20 train-test split for reproducibility.
-

8.4 Models, Metrics, and Final Results

Three models were compared: a Dummy Regressor baseline, Ridge Regression as a regularized linear benchmark, and Random Forest Regressor as a nonlinear ensemble model. Evaluation used Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 on the held-out test set.

Model	MAE	RMSE	R ²
Random Forest Regressor	€78.61	€133.76	0.5884
Ridge Regression	€85.63	€152.33	0.4662
Dummy Regressor	€134.91	€213.50	-0.0486

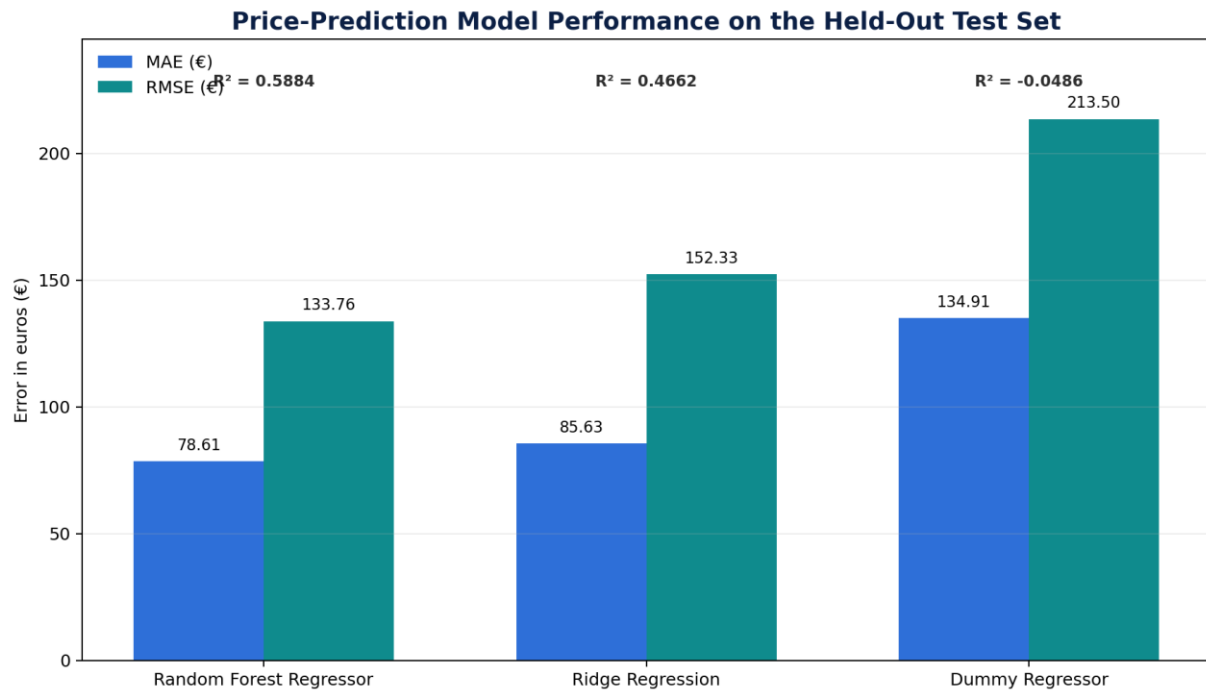


Figure 14. Held-out price-prediction model performance comparison

The Random Forest Regressor achieved the strongest held-out performance. Relative to the Dummy Regressor, it reduced MAE from €134.91 to €78.61, an improvement of approximately 41.7%. Its R² of 0.5884 indicates that roughly 58.8% of the variation in held-out listing prices was explained by the selected features within this experiment. The higher RMSE relative to MAE shows that some extreme premium listings still produce substantially larger errors.

8.5 Random Forest Feature Importance

The strongest encoded features were bedrooms, entire-home room type, longitude, minimum nights, latitude, accommodation capacity, reviews per month, bathrooms, availability_365, and unavailability_rate_proxy. Importance values describe contribution inside the fitted Random Forest model and must not be interpreted as causal effects.

Rank	Encoded Feature	Importance
1	Bedrooms	0.2714
2	Room type: Entire home/apt	0.1597
3	Longitude	0.0837
4	Minimum nights	0.0636
5	Latitude	0.0623
6	Accommodates	0.0553

7	Reviews per month	0.0351
8	Bathrooms	0.0331
9	Availability 365	0.0275
10	Unavailability-rate proxy	0.0268

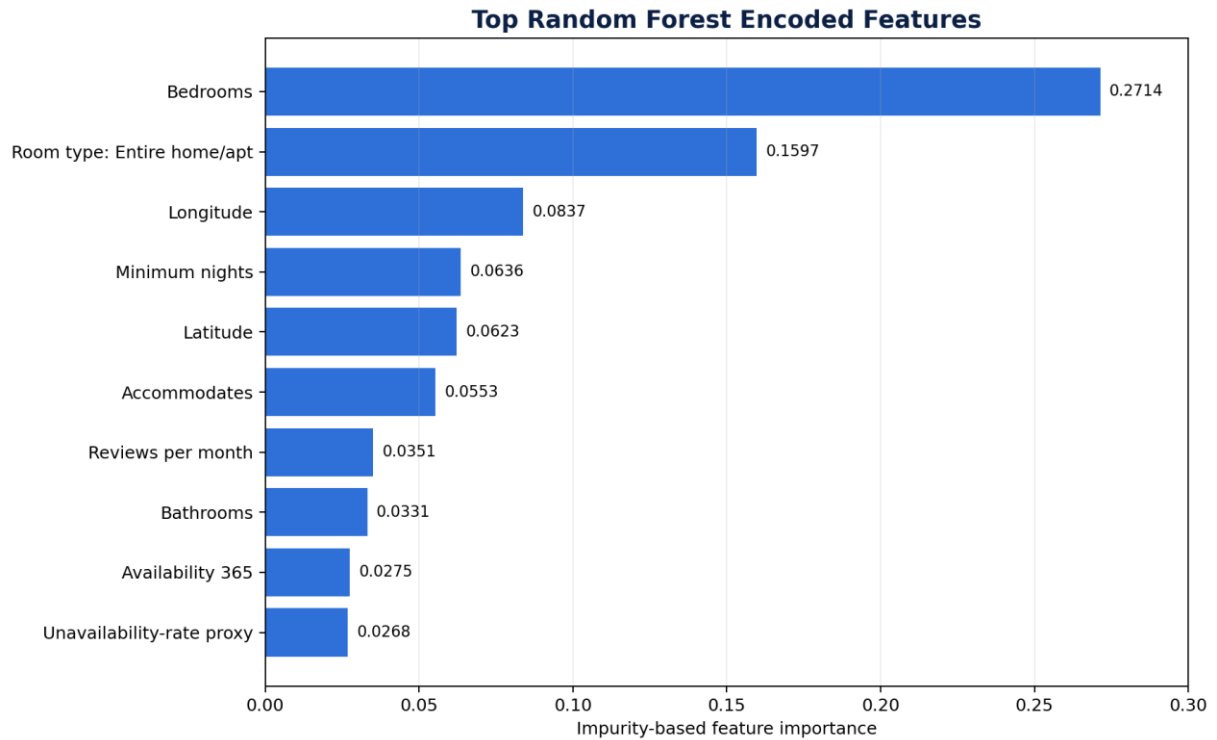


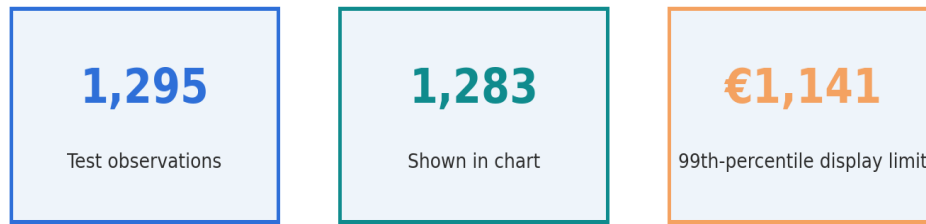
Figure 15. Top Random Forest encoded features by impurity-based importance

8.6 Actual-vs-Predicted Diagnostic Coverage

The saved actual-vs-predicted diagnostic uses the 99th percentile of actual test prices (€1,141) as a display limit for readability. The chart therefore displays 1,283 of 1,295 test observations; only 12 extreme-price points are omitted from the visual display. All 1,295 test observations remain included in MAE, RMSE, R^2 , model evaluation, and the saved prediction output.

Actual-vs-Predicted Diagnostic Coverage

Random Forest test-set visualization and evaluation coverage



12 extreme-price observations were omitted only from the scatter display for readability.

All 1,295 test observations remained included in MAE, RMSE, R^2 , model evaluation, and saved prediction outputs.

Visualization limits affect display only; they do not remove records from the model evaluation or source datasets.

Figure 16. Actual-vs-predicted diagnostic coverage and visualization limit

8.7 Data Science Interpretation and Limitations

- The model predicts available or asking listing price, not verified booked transaction price.
- The experiment does not estimate actual revenue, profitability, occupancy, or causal effects.
- Missing target prices were excluded rather than imputed.
- Extreme valid prices were retained in evaluation; percentile limits were applied only to visualizations.
- No extensive cross-validation or formal hyperparameter optimization was performed.
- Feature importance indicates predictive contribution inside the fitted model and does not establish causation.
- The experiment should be treated as a focused analytical extension rather than a production pricing system.

9. AI/ML Experiments

The completed price-prediction experiment in Section 8 is a classical supervised machine-learning experiment and represents the implemented ML work on the Airbnb dataset. No separate NLP, large-language-model, agentic, or generative-AI experiment was performed on the Airbnb data itself.

The project did not use an LLM to generate Airbnb analytical results, did not build sentiment or topic models over review comments, and did not create an agentic pipeline that automatically changes source data. This boundary is reported explicitly to distinguish implemented machine learning from AI-assisted development support.

9.1 AI Assistance Boundary

Generative AI was used only as a supporting assistant for ideas, planning, debugging guidance, review, and documentation support. The implementation, execution, testing, pipeline outputs, statistical results, machine-learning outputs, dashboard deployment, and final conclusions were validated against the actual project artifacts. A concise disclosure appears in Appendix A.

9.2 Potential Future AI/ML Work

- NLP topic or sentiment analysis of review comments, with multilingual handling and manual validation.
- LLM-assisted qualitative summarization of review corpora, evaluated against reviewed samples.
- A read-only data-quality assistant that explains validation results without modifying source data automatically.
- A natural-language interface over DuckDB with strict SQL validation and read-only execution.

10. Visualizations

Visualization quality was treated as part of analytical communication rather than decoration. Figures were selected to answer specific business questions, use readable titles and axes, prefer robust median-based summaries when prices are skewed, and include caveats where the source cannot support a stronger interpretation.

10.1 Visualization Principles

- Use medians for highly skewed price distributions.
- Apply minimum sample thresholds where rankings could be distorted by very small groups.
- Use visualization-only trimming or percentile limits without removing records from the underlying data.
- Never rename an availability proxy as occupancy.
- Never treat review count as confirmed bookings.
- Pair major figures with finding, business meaning, recommended action, and caveat.
- Keep model-diagnostic limitations explicit and separate display limits from evaluation coverage.

10.2 Supporting Figure: Full Price Distribution

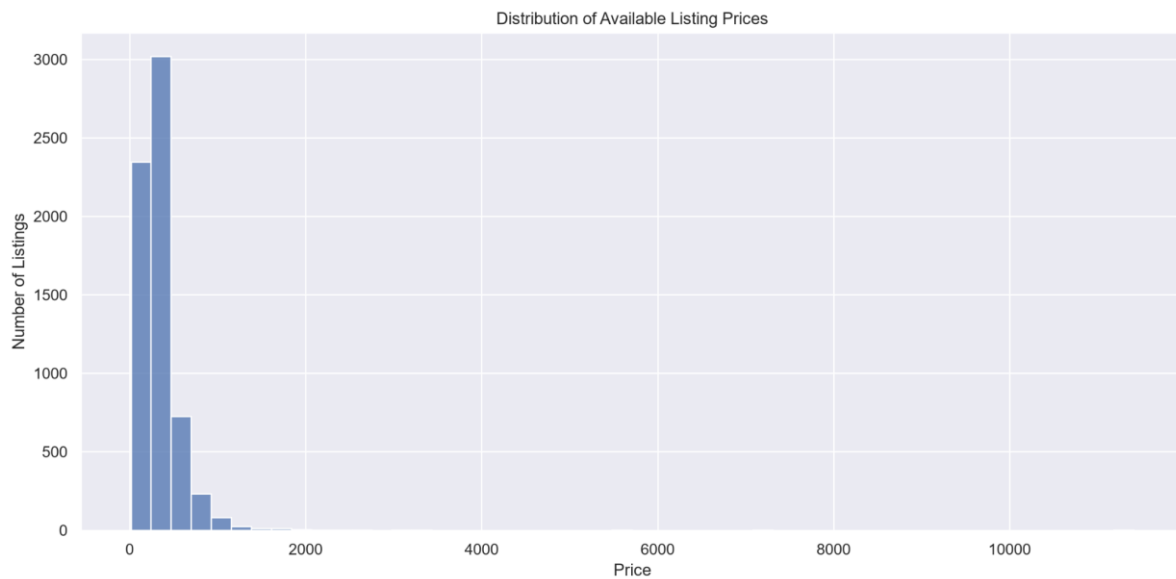


Figure 12. Full available listing price distribution

The full price distribution is extremely right-skewed, with a small number of very high-priced observations extending far beyond the main body of the data. This supports the use of medians and a separate trimmed visualization for readability while leaving the source and processed datasets unchanged.

10.3 Supporting Figure: Median Price by Host Portfolio Segment

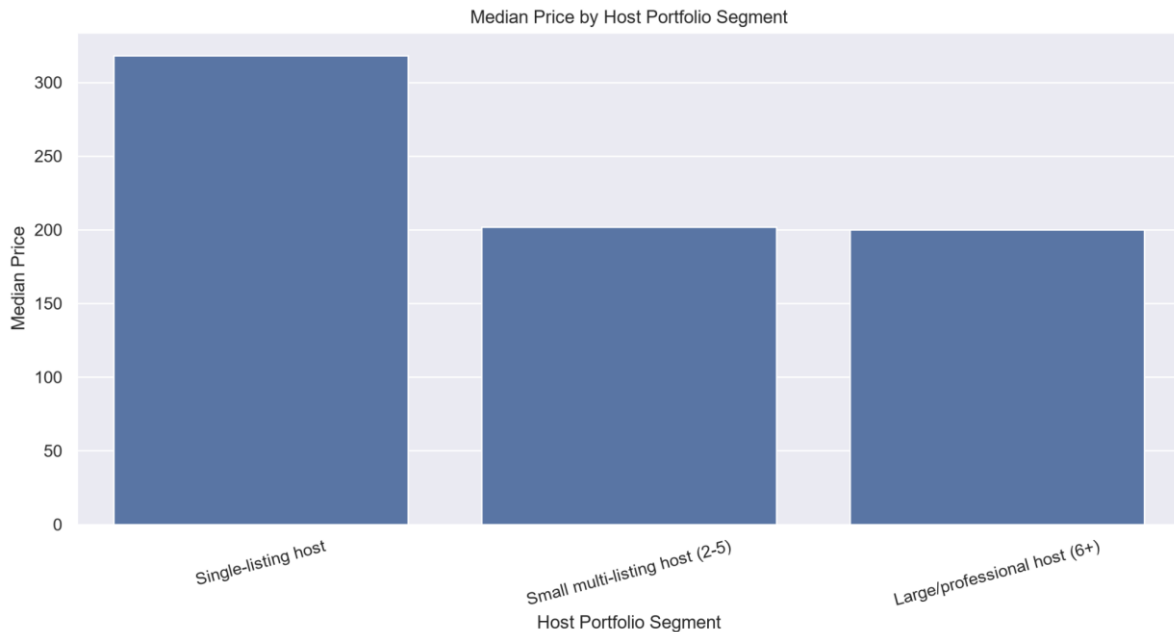


Figure 13. Median price by host portfolio segment

Single-listing hosts show a higher median price than the two multi-listing segments in this snapshot. The result should be interpreted cautiously because host segment may be confounded by room type, neighbourhood, property size, and portfolio composition. It is therefore supporting evidence rather than a standalone causal claim.

10.4 Visual Consistency and Figure Numbering

Figures are numbered sequentially across the report. The architecture diagram is Figure 1; primary EDA figures are Figures 2-9; statistical figures are Figures 10-11; supporting EDA figures are Figures 12-13; machine-learning figures are Figures 14-16; and the interactive-dashboard overview is Figure 17.

10.5 Interactive Dashboard and Live Analytical Delivery

The interactive dashboard provides a live exploration layer over validated processed outputs. It combines market KPIs, neighbourhood and room-type exploration, pricing distributions, review and availability patterns, statistical findings, machine-learning results, and engineering validation status. The dashboard intentionally does not load all raw source files directly.

Amsterdam Airbnb Market Explorer

Interactive Streamlit + Plotly dashboard built on validated processed outputs



Open the live dashboard

Streamlit Community Cloud

Filters: neighbourhood · room type · host portfolio · superhost status · price range · guest capacity

Default view preserves all 10,465 canonical listings; null values are excluded only when the corresponding numeric filter is actively narrowed.

Figure 17. Amsterdam Airbnb Market Explorer dashboard overview and live-access QR code

Dashboard Area	Purpose
Overview	High-level KPIs and summary charts
Market Explorer	Neighbourhood, room type, capacity, and price patterns
Pricing	Price distribution and segmented price comparisons
Reviews & Availability	Review activity and unavailability-rate proxy patterns
Statistics	Two completed hypothesis tests and saved output table
Machine Learning	Model comparison, feature importance, and diagnostic results
Data Engineering	Pipeline, data quality, tests, CI, and architecture

Live dashboard: <https://amsterdam-airbnb-market-explorer.streamlit.app/>

Local command: `streamlit run dashboard/app.py`

11. Business Recommendations

Recommendation	1. Segment pricing by room type
Evidence	Entire homes/apartments have a €331 median price compared with €171 for private rooms, with a large rank-biserial effect size of 0.697.
Action	Use room-type-specific price benchmarks rather than a single Amsterdam-wide benchmark.
Caveat	The statistical test identifies a distributional difference but does not prove that room type alone causes the price gap.

Recommendation	2. Use neighbourhood-specific benchmarking
Evidence	Median prices and listing supply vary substantially across Amsterdam neighbourhoods.
Action	Compare listings within the same neighbourhood and room-type segment before making pricing or market-entry decisions.
Caveat	Neighbourhood effects are associated with property mix and other listing characteristics.

Recommendation	3. Include accommodation capacity in price comparisons
Evidence	Median price generally rises with guest capacity, and size-related variables show meaningful positive relationships with price.
Action	Benchmark properties against similar capacities and avoid comparing one-person listings with large-group accommodation.
Caveat	Group sample sizes should be checked for very high capacities.

Recommendation	4. Prefer medians for broad pricing summaries
Evidence	The full price distribution is extremely right-skewed and affected by premium observations.
Action	Use medians and quantiles for market benchmarks; use means only with clear context.
Caveat	Display trimming should never be confused with removing valid source records.

Recommendation	5. Treat calendar unavailability only as a proxy
Evidence	Unavailable dates may reflect bookings, host blocks, maintenance, personal use, regulation, or another reason.

Action	Never present unavailability_rate_proxy as verified occupancy, confirmed demand, or revenue.
Caveat	The source data does not reveal the reason a date is unavailable.

Recommendation	6. Do not use superhost status as a standalone quality signal
Evidence	The review-score difference is statistically significant but practically small: 4.90 vs 4.94 medians and effect size -0.158.
Action	Combine superhost status with review volume, response behaviour, consistency, and listing characteristics.
Caveat	Large samples can make very small differences statistically detectable.

Recommendation	7. Use the Random Forest as an analytical benchmark, not an automated pricing engine
Evidence	The Random Forest achieved the best held-out performance with MAE €78.61, RMSE €133.76, and R^2 0.5884.
Action	Use predictions as one contextual benchmark alongside neighbourhood, room type, capacity, market knowledge, and business rules.
Caveat	The target is available listing price, not verified transaction price, revenue, profitability, or causal effect.

Recommendation	8. Use the live dashboard for segmented exploration, not as a substitute for source caveats
Evidence	The seven-tab dashboard exposes validated engineering, market, statistical, and model outputs with six filter dimensions.
Action	Use the dashboard to explore comparable listing segments and communicate findings consistently.
Caveat	Interactive convenience does not remove the limitations of missing prices, proxies, observational data, or single-city scope.

12. Cross-City Comparisons

Not applicable to the implemented project scope.

The final project analyzes Amsterdam only. No cross-city comparison is presented, and no claims are generalized to other Airbnb markets. This decision is consistent with the depth-first strategy used throughout the project.

A cross-city extension would require schema harmonization, source-snapshot alignment, currency handling where relevant, city-specific data-quality rules, geographic comparability, and careful control for differences in market size and property mix. Adding another city without those safeguards could produce misleading comparisons.

12.1 Future Cross-City Design

- Parameterize city-specific raw paths and configuration.
- Validate schemas before concatenation or comparison.
- Track source snapshot dates explicitly.
- Normalize currencies or restrict comparisons to equivalent units.
- Use comparable room-type and capacity segments.
- Report within-city and between-city variation separately.
- Evaluate whether predictive models remain stable across cities.

13. Limitations & Caveats

Limitation	Implication
Single-city scope	The report describes Amsterdam only and does not generalize findings to other markets.
Snapshot data	The analysis reflects one source snapshot rather than a full longitudinal history of listing changes.
Scraped public data	Source inconsistencies, delayed updates, or missing attributes may remain.
Missing prices	3,994 canonical listings lack a valid target price, reducing sample size for pricing analysis and modeling.
Detailed-source coverage	96 canonical summary listings are absent from the detailed listing source.
Calendar proxy limitation	Unavailable dates are not verified bookings and are never reported as true occupancy.
Review proxy limitation	Review counts are not complete booking counts.
No verified revenue or transaction prices	The project does not calculate actual host earnings, booked prices, or profitability.
Observational statistics	Significant differences and correlations do not establish causation.
Focused ML evaluation	No extensive cross-validation, broad hyperparameter search, or cross-city model validation was performed.
Model target limitation	The model predicts available listing price, not verified booked price, revenue, or causal effect.
Feature importance limitation	Impurity-based importance reflects model contribution and does not prove causality.
Batch processing	The engineering pipeline is full-batch rather than incremental.
No production orchestration	Scheduling, retry policies, alerts, and workflow observability are not implemented.
Dashboard deployment scope	The Streamlit dashboard is deployed, but the full engineering pipeline is not cloud-native or production-operated.

14. Future Improvements

Improvement	Value	Priority
Multi-city pipeline support	Generalize configuration and validation across markets.	Future
Automated schema harmonization	Detect and reconcile source-schema changes.	Future
Incremental processing	Use watermarks, partitions, snapshots, or change detection instead of full batch rebuilds.	Future
Full cloud-native pipeline deployment	Move ingestion, transformation, storage, and scheduling to managed infrastructure.	Future
Workflow orchestration	Add retries, dependencies, schedules, backfills, and monitoring.	Future
Dockerization	Improve environment portability and dependency isolation.	Future
Advanced data-quality framework	Add contracts, trend checks, anomaly detection, and observability.	Future
Advanced geospatial analysis	Add choropleths, spatial clustering, and interactive boundary layers.	Future
Additional statistical hypotheses	Extend carefully with multiple-comparison awareness.	Future
Cross-validation and hyperparameter optimization	Improve robustness of the focused price-prediction experiment.	Future
Permutation or grouped feature importance	Provide more robust model interpretation than impurity importance alone.	Future
Production model serving and monitoring	Add controlled inference, drift monitoring, and model lifecycle management only if required.	Future
Historical snapshot tracking	Support point-in-time changes in listings, hosts, prices, and availability.	Future

The completed machine-learning experiment, GitHub Actions CI, interactive dashboard, and live dashboard deployment are no longer future improvements. The most defensible next extensions are multi-city generalization, incremental processing, stronger cloud-native pipeline operations, and deeper validation of model robustness.

15. Reflection

The strongest lesson from this project was that reliable analytics depend on understanding dataset grain, relationships, source coverage, and missingness before performing analysis. Treating listings.csv as the canonical population and preserving 96 summary-only listings shows how a single join decision can change the truth represented by a dataset.

A second important lesson was that not every data-quality issue should block a pipeline. The PASS / WARNING / FAIL framework made it possible to distinguish critical structural errors from known source limitations. Missing prices, an empty neighbourhood_group field, repeated summary review dates, and one missing reviewer name were documented transparently rather than hidden, deleted, or artificially repaired.

The 8 GB RAM constraint became an engineering design input rather than a hidden weakness. Using Pandas for smaller sources, DuckDB for large detailed tables, Parquet for processed outputs, listing-level aggregation before joins, and sequential stage execution allowed the complete Amsterdam snapshot to run without downsampling the core datasets.

The statistical work reinforced the importance of practical significance. Hypothesis 1 showed a large pricing difference between entire homes and private rooms. Hypothesis 2 showed that a very small effect can still produce an extremely small p-value when sample sizes are large. Effect size and business context therefore matter as much as statistical significance.

The machine-learning extension reinforced the importance of baselines, leakage prevention, honest target definition, and held-out evaluation. The Random Forest clearly outperformed the Dummy baseline, but its results remain bounded by missing target prices, extreme premium listings, observational data, and the fact that available price is not verified transaction price.

The dashboard added a final lesson about delivery: validated outputs become more useful when stakeholders can explore them interactively, but interactivity must not weaken caveats. The deployed app therefore keeps proxy limitations, statistical interpretation, and model limitations visible alongside the charts.

The main trade-off remains breadth. Multi-city analysis, full cloud-native pipeline execution, orchestration, incremental processing, advanced geospatial work, and production model serving were intentionally deferred. That decision protected the quality of the completed pipeline, tests, CI, statistics, machine learning, dashboard, documentation, and report.

Final reflection: the project is intentionally focused rather than broad, but it is complete across engineering, analytics, statistical reasoning, predictive experimentation, automated verification, and live interactive delivery - while remaining explicit about what the data can and cannot prove.

Appendix A. AI Usage Disclosure

This appendix briefly explains how generative AI was used during the Amsterdam Airbnb Data Engineering & Analytics Challenge.

A.1 AI Tool Used

ChatGPT - GPT-5.5 Thinking

A.2 How AI Was Used

AI was used as a supporting tool to help manage, organize, implement, and review the project. The assistance mainly included:

- Generating ideas for how to approach the assignment.
- Helping plan the project structure and implementation order.
- Providing guidance when writing and improving Python code.
- Assisting with code review, debugging, syntax errors, logic issues, and implementation problems.
- Suggesting possible solutions when technical problems occurred.
- Giving guidance for data engineering, data analysis, statistical testing, machine learning, automated testing, dashboard development, and documentation.
- Helping review and improve technical explanations, README content, project documentation, and the final report.

AI-generated code suggestions were not accepted automatically. The code was reviewed, adapted where necessary, executed locally, tested against the actual project data, and validated using pipeline outputs, automated tests, statistical results, machine-learning outputs, and the deployed dashboard.

AI was used only as a supporting assistant. The final project was implemented, executed, tested, reviewed, and validated using the actual datasets, source code, pipeline outputs, DuckDB warehouse results, statistical findings, machine-learning results, dashboard deployment, GitHub Actions CI, and automated tests.

A.3 Final Disclosure Statement

Generative AI was used to provide ideas, coding guidance, code review, debugging support, technical guidance, documentation assistance, and help with managing the overall project workflow.

Final responsibility for the implementation, code, analysis, testing, documentation, interpretation, and conclusions remains with the project author.

Appendix B. Reproducibility & Validation Evidence

B.1 Reproducibility Commands

Task	Command
Clone repository	<code>git clone https://github.com/PabodaFdo/airbnb-data-engineering-challenge.git</code>
Create environment	<code>python -m venv .venv</code>
Install dependencies	<code>pip install -r requirements.txt</code>
Run pipeline	<code>python run_pipeline.py --city amsterdam</code>
Run tests	<code>python -m pytest tests/ -v</code>
Compile project	<code>python -m compileall -q run_pipeline.py src tests experiments dashboard</code>
Run ML experiment	<code>python experiments/price_prediction.py</code>
Run dashboard locally	<code>streamlit run dashboard/app.py</code>

B.2 Key Project Files

- `run_pipeline.py`
- `src/profile.py`
- `src/validate.py`
- `src/clean.py`
- `src/enrich.py`
- `src/build_warehouse.py`
- `sql/analytical_queries.sql`
- `tests/test_data_quality.py`
- `experiments/price_prediction.py`
- `dashboard/app.py`
- `dashboard/charts.py`
- `dashboard/components.py`
- `dashboard/data_loader.py`
- `docs/architecture_diagram.png`
- `docs/assumptions.md`
- `docs/decision_log.md`
- `docs/completed_work.md`
- `docs/incomplete_work.md`
- `docs/ai_usage_disclosure.md`

B.3 Final Verified Engineering, Analytical, and Delivery Results

Metric	Result
Source datasets profiled	7 / 7
Source validation	83 PASS / 7 WARNING / 0 FAIL
Canonical listings preserved	10,465
Detailed-source matches	10,369

Summary-only listings retained	96
Review events reconciled	545,162
Calendar rows reconciled	3,819,725
Warehouse validation	17 PASS / 0 FAIL
Automated tests	13 passed / 0 failed
GitHub Actions CI	Passing
EDA visualizations	10
Statistical hypotheses	2
ML models compared	3
Best model	Random Forest Regressor
Best model MAE	€78.61
Best model RMSE	€133.76
Best model R ²	0.5884
Dashboard tabs	7
Dashboard deployment	Live on Streamlit Community Cloud
Live dashboard	https://amsterdam-airbnb-market-explorer.streamlit.app/
Full pipeline runtime	Approximately 1-2 minutes depending on system conditions and disk caching

References & Data Attribution

Inside Airbnb. Public Airbnb data used for the Amsterdam market snapshot. Data source: insideairbnb.com/get-the-data/

Project repository: <https://github.com/PabodaFdo/airbnb-data-engineering-challenge.git>

(dev branch used as the reporting source of truth during final report preparation).

Live dashboard: <https://amsterdam-airbnb-market-explorer.streamlit.app/>

Primary implementation technologies: Python, Pandas, DuckDB, Parquet, SciPy, scikit-learn, Matplotlib, Seaborn, Plotly, Streamlit, Pytest, Jupyter, and GitHub Actions.

Statistical methodology: Mann-Whitney U tests with rank-biserial effect sizes, interpreted alongside group medians, sample sizes, practical significance, and limitations.

Machine-learning methodology: held-out 80/20 train-test evaluation with leakage prevention, baseline comparison, log-transformed target modeling, and MAE, RMSE, and R^2 reported in original price units.